

1) AERIS — HOW THE SOFTWARE WORKS

Full practical explanation of the current software logic

Use this as your **internal architecture guide / operator note**.

AERIS SOFTWARE EXPLANATION

What the system is, how it works, and how the parts connect

AERIS is a modular aircraft-design software backbone built around one core idea:

separate concerns cleanly

so geometry, sampling, datasets, and later aero can evolve independently without turning the repo into spaghetti.

This means the software is not organized as one giant script that “does everything.”

Instead, it is organized into **layers**.

1. HIGH-LEVEL ARCHITECTURE

The software currently has four important active layers:

A. Framework layer

These are the generic abstractions that define how the system behaves.

Examples:

- `src/aeris/geometry/base.py`
- `src/aeris/geometry/registry.py`
- `src/aeris/geometry/config_resolver.py`
- `src/aeris/dataset/sampling/base.py`
- `src/aeris/dataset/sampling/registry.py`
- `src/aeris/dataset/sampling/resolver.py`

These files do **not** contain BWB math.

They define:

- what a generator is
- how generators are selected
- what a sampler is
- how samplers are selected

This is the **architecture spine**.

B. Implementation layer

These are the actual aircraft-family-specific and sampler-specific implementations.

Examples:

- `src/aeris/generators/bwb_segmented_v1/`
- `src/aeris/dataset/sampling/samplers/lhs_v1.py`

This is where the real math and domain logic lives.

For example:

- BWB planform construction
- section geometry generation
- exports
- plotting
- validation
- LHS sampling over current BWB parameter bounds

This layer is **modular by family/version**.

C. Pipeline / orchestration layer

These files do not define geometry formulas.

They **orchestrate** workflows.

Examples:

- `src/aeris/pipeline/geometry_run.py`
- `src/aeris/dataset/dataset_run.py`

These are the “workflow engines.”

They:

- load config
- resolve generator
- resolve sampler
- run the correct workflow
- save artifacts
- save manifests
- handle failures

Think of them as **conductors**, not musicians.

D. CLI layer

This is the user interface to the software.

Examples:

- `src/aeris/cli.py`
- `src/aeris/commands/*.py`

This layer:

- receives terminal commands
- validates arguments
- passes them to the correct pipeline function

It should remain thin and boring.

That is good.

2. CURRENT ACTIVE MODULAR SYSTEMS

A. Geometry generators

Current active generator:

- `bwb_segmented_v1`

This is a self-contained geometry family in:

`src/aeris/generators/bwb_segmented_v1/`

That folder contains:

- `generator.py`
- `params.py`
- `sampling.py`
- `planform.py`
- `sections.py`
- `case.py`
- `export.py`
- `plotting.py`
- `validation.py`
- `validators.py`
- `aerosandbox_adapter.py`

This means:

- all BWB-specific geometry logic lives together
- the framework does not need to know the math details

That is one of the biggest architectural wins.

B. Dataset samplers

Current active sampler:

- `lhs_v1`

Sampler framework:

`src/aeris/dataset/sampling/`

Current implementation:

src/aeris/dataset/sampling/samplers/lhs_v1.py

This means:

- dataset generation no longer assumes LHS forever
- future samplers can be added without rewriting the dataset pipeline

Examples of future samplers:

- random_v1
 - sobol_v1
 - halton_v1
 - adaptive_v1
 - active_learning_v1
-

C. Aero system

Current state:

- framework shell exists
- operational modular aero workflow not implemented yet

So aero is **prepared structurally**, but not yet a finished workflow.

That is okay for now.

3. CORE OPERATIONAL LOGIC

There are two main working production paths.

PATH 1 — Single geometry generation

This is triggered by:

aeris geometry generate --config ...

What happens internally

1. YAML config is loaded
2. generator is resolved from config

3. generator config object is built and validated
4. one design sample is produced using seed
5. geometry is built deterministically from that sample
6. artifacts are exported:
 - geometry summary JSON
 - control points CSV
 - planform sections CSV
 - section 3D CSV
 - optional plot
7. run manifest is written
8. logs are written

Important design rule

The RNG is used only to create the **explicit sample**.
After that:

geometry = deterministic function(sample)

That is critical for:

- reproducibility
- ML datasets
- certification-style traceability

PATH 2 — Dataset generation

This is triggered by:

```
aeris dataset generate --config ... --n ...
```

What happens internally

1. YAML config is loaded
2. geometry generator is resolved

3. sampler is resolved
4. generator config object is built
5. sampler generates n explicit design samples
6. each sample is passed into generator deterministic realization
7. for each geometry:
 - geometry artifacts are written
 - metadata row is appended
 - failure row is appended if needed
8. dataset manifest is written

Important design rule

Sampling and geometry are separated:

sampler -> explicit samples

generator -> deterministic geometry realization

That is exactly the correct software logic.

4. CONFIGURATION LOGIC

Configs currently live under:

configs/

Important ones:

- configs/geometry/wing_bwb.yaml
- configs/geometry/baseline_bwb.yaml
- configs/smoke/dev.yaml
- configs/smoke/bad.yaml

What config controls

A config currently contains:

- geometry generator family/version
- generator controls
- planform bounds

- section bounds
- output settings
- dataset sampling settings

Current logic

The software reads raw YAML, then:

1. resolves the generator ID
2. gives raw config to the generator
3. generator builds its own typed config object
4. generator validates that config

This is important because:

- generator owns its own config philosophy
- pipeline does not hardcode BWB rules anymore

That is much better than the old design.

5. FILE OUTPUT LOGIC

Single-case output

Stored under:

data/runs/<timestamp>_geometry_<config_name>/

Contains:

- logs
- copied input config
- artifacts
- manifest

Dataset output

Stored under:

data/datasets/<dataset_name>/

Contains:

- geometry cases
- metadata CSV
- failures CSV
- manifest
- copied config
- resolved config JSON
- logs

Debug visual output

Stored under things like:

data/debug/

Examples:

- data/debug/wing_bwb
 - data/debug/baseline_bwb
 - data/debug/visualization_runs/baseline
-

6. VALIDATION LOGIC

There are several kinds of validation in the system.

A. Config validation

Checks whether YAML-derived generator config is valid.

Examples:

- bounds exist
- min/max relationships are valid
- geometric parameters are consistent

B. Geometry validation

Checks planform and section geometry results.

Examples:

- no degenerate spans
- no invalid section arrays
- no broken geometry outputs

C. Architecture / reproducibility validation

Scripts like:

- `check_architecture_integrity.py`
- `check_metadata_regeneration.py`

These validate:

- deterministic behavior
- reproducibility
- dataset consistency
- metadata correctness

These are not decoration. They are part of the trust model of the software.

7. WHY THE SOFTWARE IS STRUCTURED THIS WAY

Because your future project needs all of these things simultaneously:

- research reproducibility
- ML dataset traceability
- multiple geometry families
- multiple sampling strategies
- future solver plug-ins
- product-grade maintainability

If the repo stayed as:

- one geometry folder
- one sampling method
- scripts calling whatever directly

then adding the next generator or sampler would become painful fast.

The current architecture avoids that.

8. WHAT IS STILL TRANSITIONAL

Be honest about the remaining debt.

Still not fully complete:

- aero workflow not yet fully integrated
- some dataset internals still carry BWB assumptions
- some scripts are family-specific developer tools, not generic architecture clients
- baseline config path needed extra cleanup
- some naming/documentation debt still exists

This is normal. It is now **bounded debt**, not chaos.

9. HOW TO THINK ABOUT THE SYSTEM

The best mental model is:

AERIS = modular factory

Framework defines machine interfaces

- geometry generator interface
- dataset sampler interface

Implementations are plug-in machines

- bwb_segmented_v1
- lhs_v1

Pipelines run production lines

- single geometry line
- dataset line

CLI is the control panel

- start geometry
- start dataset
- inspect dataset
- run smoke tests

That's the software.

10. WHAT TO DO WHEN ADDING NEW THINGS

New geometry family

Add:

src/aeris/generators/<new_family_version>/

Implement:

- config build
- validation
- sampling for one sample
- deterministic realization
- export/summary/plotting as needed

Register it.

Do **not** modify pipelines unless absolutely necessary.

New dataset sampler

Add:

src/aeris/dataset/sampling/samplers/<sampler_name>.py

Implement sampler interface.

Register it.

Do **not** modify dataset pipeline logic.

New aero solver

Later, do the same pattern under:

`src/aeris/aero/solvers/`